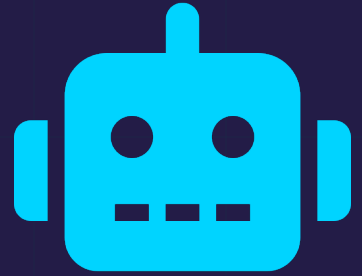


Building Your

AI Dream Team with CrewAI



Orchestrating Autonomous AI Agents Like a High-Performing Company

Software Engineers

AI Engineers

Solution Architects

Technical Managers

You're the CEO. The deadline is tomorrow.

01



Research the Market

...all by yourself?

02



Analyze Competitors

...all by yourself?

03



Write the Report

...all by yourself?

04



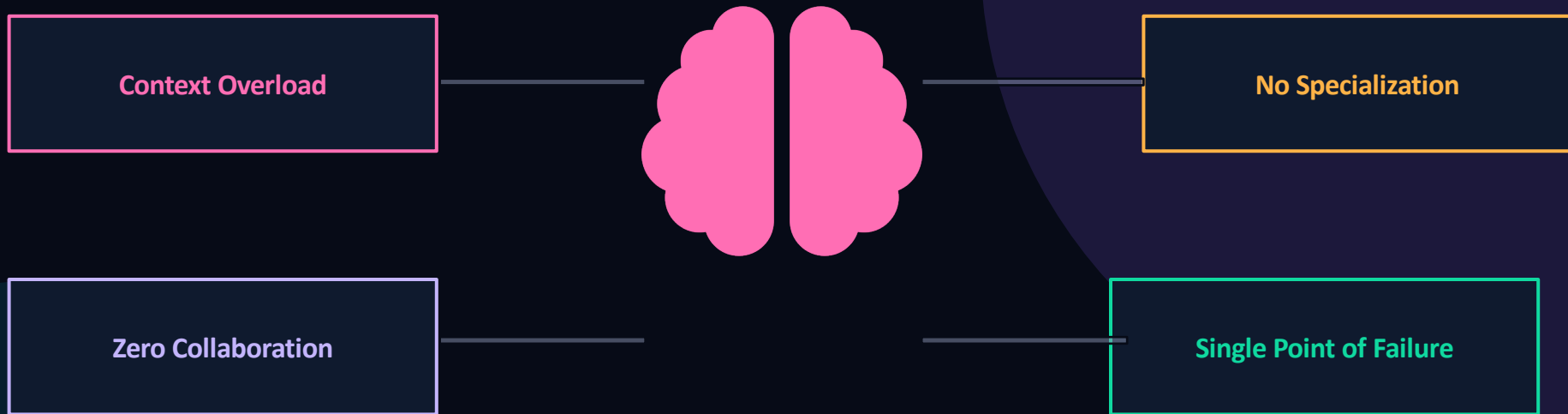
Create a Presentation

...all by yourself?



What if you could hire an AI Dream Team to do ALL of this for you?

One Brain. Infinite Demands.



Company Analogy: One employee trying to be the CEO, Researcher, Analyst, Writer, and Developer — all at once. Nobody wins.

Meet CrewAI

CrewAI is an open-source Python framework for orchestrating autonomous AI agents that collaborate to solve complex, real-world tasks.



Research Agent

Finds & gathers data



Analyst Agent

Processes & interprets

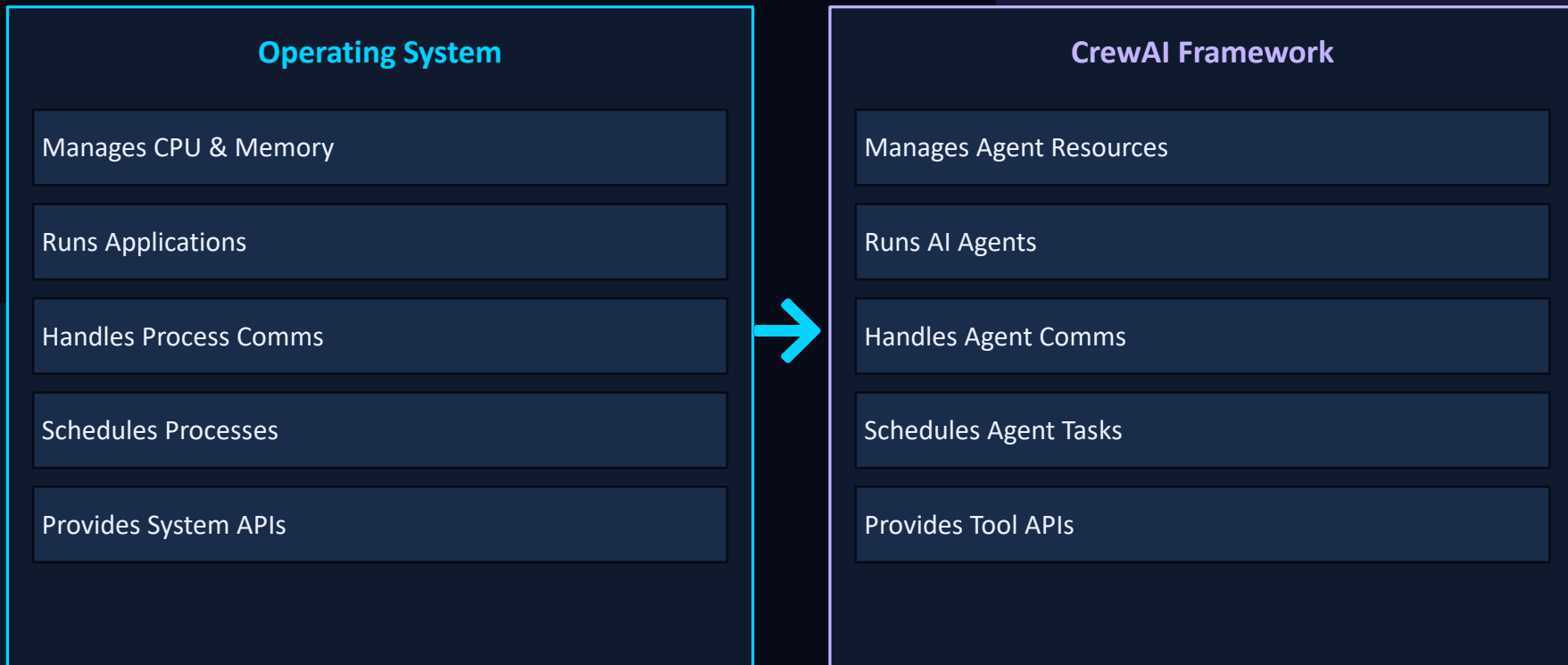


Writer Agent

Creates deliverables

★ github.com/crewAIInc/crewAI | Open Source | Python | `pip install crewai`

CrewAI = Operating System for AI Agents



Just as an OS abstracts hardware complexity, CrewAI abstracts the complexity of multi-agent coordination.

5 Reasons CrewAI Changes Everything

01

Specialization

Each agent is an expert in ONE domain — sharper, faster, better



02

Delegation

Tasks flow automatically to the best-suited agent



03

Collaboration

Agents share context, results, and memory across a pipeline



04

Scalability

Add agents as complexity grows — no re-architecture needed



05

Automation

Complex multi-step workflows run end-to-end with no manual steps



Where CrewAI Shines Today



Market Research

Auto-scrape, synthesize & report competitive intel



Financial Analysis

Multi-source data pipelines + AI-generated insights



DevOps Automation

Automated CI/CD, monitoring, alerting & remediation



Incident Response

Detect, diagnose & remediate production incidents 24/7



Content Creation

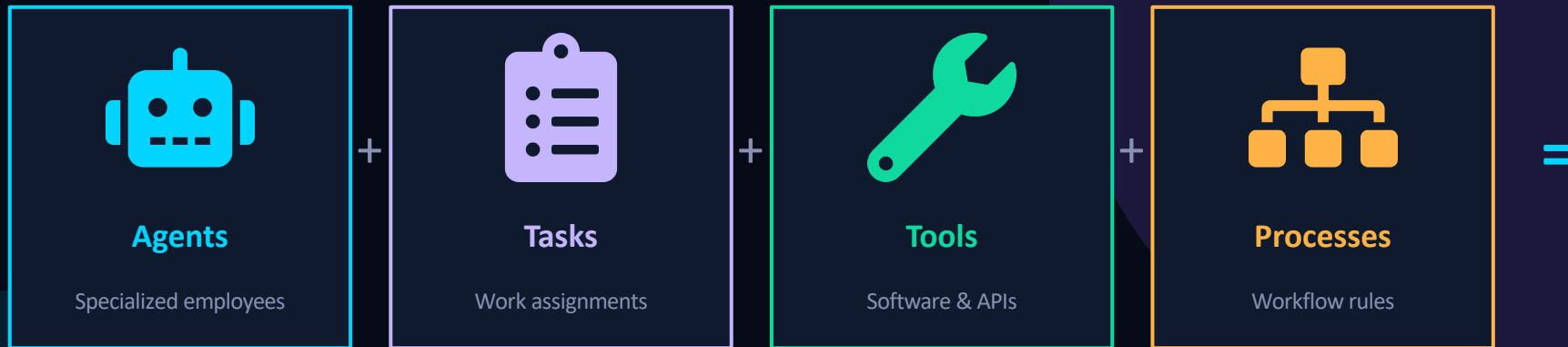
Research → Draft → Edit → Publish in one automated flow



Cloud Assessments

Multi-account AWS/Azure/GCP audits with automated reports

The CrewAI Blueprint



CREW

Your autonomous AI workforce — orchestrated, collaborative, and mission-driven

Agents — Your AI Specialists

An Agent is an autonomous AI entity with a specific role, goal, backstory, and the tools needed to complete its assigned work.



Research Analyst

Gather & synthesize market intelligence



Content Writer

Transform raw data into compelling narratives



Quality Reviewer

Validate outputs, fact-check, ensure quality



Solution Architect

Design technical blueprints & architecture

Anatomy of a CrewAI Agent

role

Job Title

goal

KPIs & Objectives

backstory

Resume & Experience

tools

Laptop & Software

llm

Brain / Intelligence

memory

Meeting Notes

allow_delegation

Management Authority

max_iter

Work Hours Cap

verbose

Audit Trail Logging

max_rpm

API Rate Limiter

Creating an Agent in Python

```
from crewai import Agent
from crewai_tools import SerperDevTool

# Instantiate a specialized Research Agent
research_agent = Agent(
    role = "Senior Market Research Analyst",
    goal = "Uncover cutting-edge AI trends and competitive insights",
    backstory = (
        "You are a world-class analyst with 10 years of experience tracking"
        "technology trends. Known for turning noise into actionable signal."
    ),
    tools = [SerperDevTool()],
    llm = "gpt-4o",
    memory = True,
    verbose = True,
    allow_delegation = True,
    max_iter = 15,
    max_rpm = 10
)
```

Tools — The Agent's Toolkit

Tools are Python functions or API wrappers that agents call to interact with the outside world — web, databases, code, files, and more.



Web Search

```
SerperDevTool()
```



Database

```
PostgreSQLTool()
```



Code Executor

```
CodeInterpreterTool()
```



REST APIs

```
CustomAPITool()
```



File I/O

```
FileReadTool()
```



LLM Chain

```
LLMTool(model='gpt-4o')
```

 Analogy: Tools = The laptop, CRM, browser, calculator, and APIs your employee uses to do their job.

Tasks — The Work Assignments

A Task defines WHAT needs to be done, WHO does it, WHAT tools are available, and WHAT a successful output looks like.

description

Detailed natural-language instructions for the agent

tools

Override the agent's default tools for this task

expected_output

Exact format/criteria — what success looks like

context

Results from prior tasks fed as context automatically

agent

The assigned agent responsible for this task

output_file

Automatically save task output to a file



Analogy: Tasks = The manager's brief — clear instructions, measurable deliverables, assigned team member.

Creating Tasks in Python

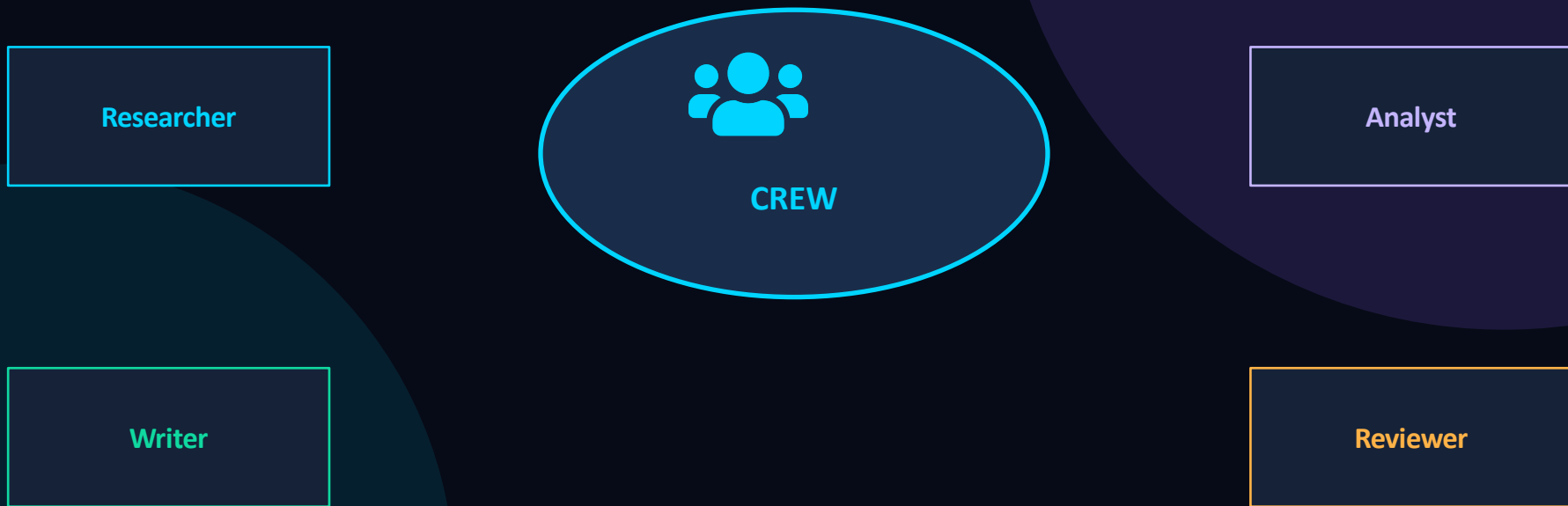
```
from crewai import Task

# Task 1 – Assigned to the Research Agent
research_task = Task(
    description="""
        Investigate the top 5 AI agent frameworks available in 2025.
        For each framework document: name, key features, primary use-cases,
        GitHub stars, license, and main limitations.
    """,
    expected_output="A structured markdown table comparing all 5 frameworks.",
    agent=research_agent,
    output_file="framework_research.md"
)

# Task 2 – Assigned to the Writer Agent, uses research output as context
write_task = Task(
    description="Write a 500-word blog post based on the research findings.",
    expected_output="A ready-to-publish blog post in markdown format.",
    agent=writer_agent
)
```

The Crew — Your AI Company

A Crew is the orchestrating layer — it combines agents, assigns tasks, manages process flow, and drives the team toward a shared mission.



```
crew = Crew(agents=[researcher, analyst, writer, reviewer], tasks=[t1, t2, t3, t4], process=Process.sequential)
```

Processes — Workflow Rules

Processes define HOW agents coordinate: who acts first, how outputs flow, and what triggers the next agent's work.



Sequential

"Assembly Line"

Agents execute in fixed order, each passing output to the next



Hierarchical

"Corporate Org"

A manager agent assigns and coordinates worker agents dynamically



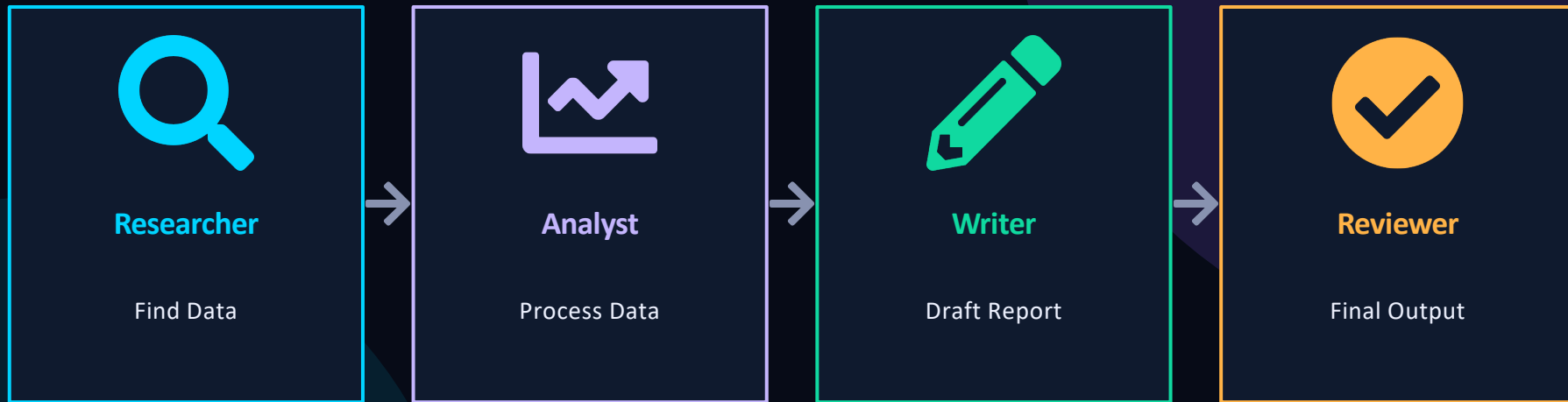
Consensual

"Board Meeting"

Agents collaborate, discuss, and vote to reach group consensus

Sequential Process

Like a manufacturing assembly line — each agent passes finished work to the next



```
from crewai import Process
crew = Crew(agents=[researcher, analyst, writer, reviewer],
            tasks=[t1, t2, t3, t4],
            process=Process.sequential, # DEFAULT – simplest to use
            verbose=True)
```

Hierarchical Process

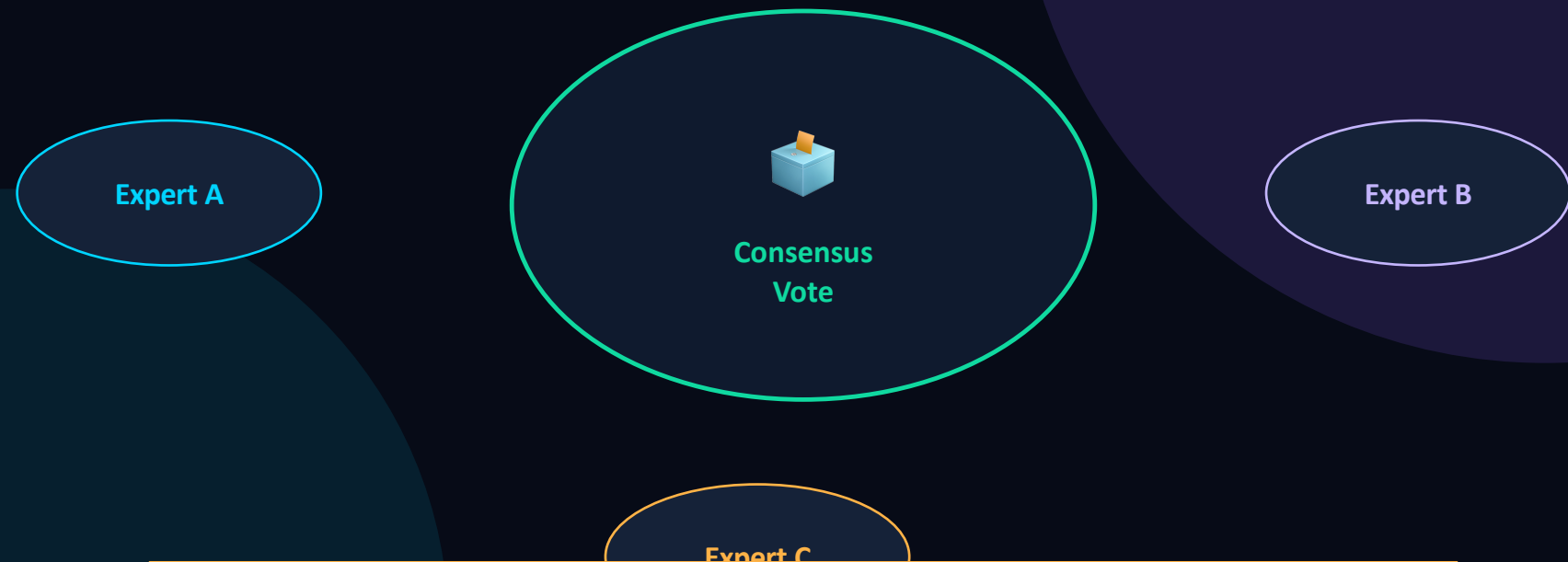
Like a corporate org chart — a manager agent delegates dynamically to worker agents



```
crew = Crew(agents=[researcher, analyst, writer],
            tasks=[complex_task_1, complex_task_2],
            process=Process.hierarchical,
            manager_llm="gpt-4o" # Manager must be more capable)
```

Consensual Process

Like a board meeting — agents present positions, deliberate, and vote toward a shared conclusion



⚠️ Consensual process is on the CrewAI roadmap — check docs.crewai.com for latest availability.

Choosing the Right Process

Scenario	Sequential	Hierarchical	Consensual
Linear pipeline (A → B → C)	✓	⚠	✗
Complex multi-step task routing	⚠	✓	⚠
Expert review & validation	✗	⚠	✓
Simple 2-3 agent workflows	✓	✗	✗
Dynamic task assignment	✗	✓	⚠
Debate, reasoning, high-stakes output	✗	⚠	✓
Beginner / first CrewAI project	✓	✗	✗

Building an Effective AI Team

01



Define Crystal-Clear Roles

Every agent has ONE specialized purpose. Ambiguity destroys agent performance.

02



Eliminate Role Overlap

If two agents have the same goal, they'll compete and loop. Redesign immediately.

03



Assign Precise Tools

Give agents exactly the tools they need. Over-tooling creates confusion.

04



Set Measurable Outputs

Specific expected_output prevents vague, unusable responses from agents.

05



Delegate Carefully

Only senior/manager agents get allow_delegation=True. Others are workers only.

Proven Team Compositions



Complete CrewAI Application

```
from crewai import Agent, Task, Crew, Process
from crewai_tools import SerperDevTool

# — AGENTS —
researcher = Agent(role='Senior Researcher', goal='Find AI trends in 2025',
    backstory='Expert analyst tracking emerging technologies.',
    tools=[SerperDevTool()], llm='gpt-4o', memory=True, verbose=True)

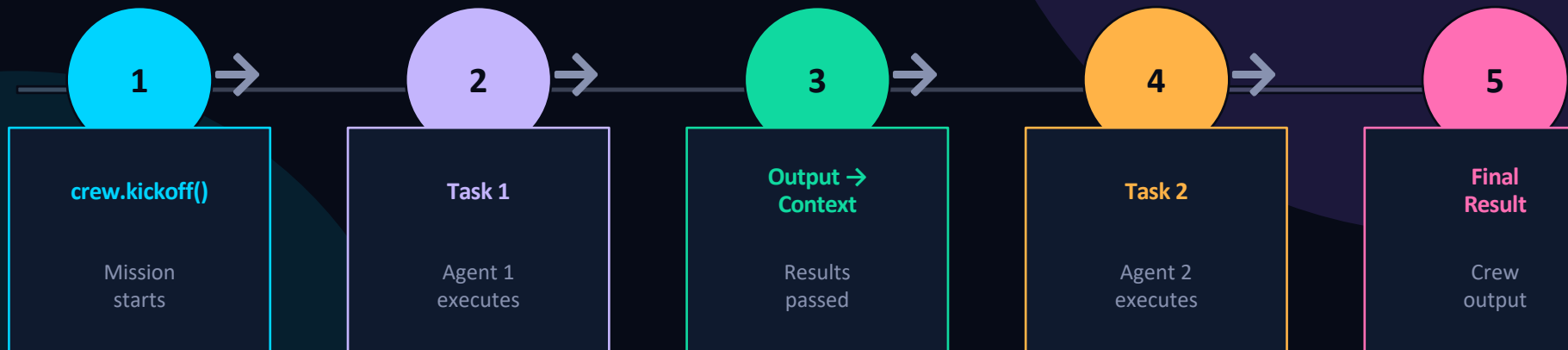
writer = Agent(role='Tech Writer', goal='Write compelling technical blog posts',
    backstory='Award-winning writer bridging AI and general audiences.',
    verbose=True)

# — TASKS —
research_task = Task(
    description='Research top 5 AI agent frameworks released in 2025.',
    expected_output='Markdown table: Framework | Stars | Key Feature | Use-case',
    agent=researcher, output_file='research.md')

write_task = Task(
    description='Write a 600-word blog post from the research findings.',
    expected_output='Ready-to-publish markdown blog post with intro and CTA.',
    agent=writer)

# — CREW KICKOFF —
crew = Crew(agents=[researcher, writer], tasks=[research_task, write_task],
    process=Process.sequential, verbose=True)
result = crew.kickoff()
print(result.raw) # The final published blog post!
```

How Data Flows Through a Crew



Each task's full output becomes the context input for the next task automatically — no manual wiring needed.

How Agents Remember

CrewAI provides four layers of memory so agents can remember, recall, and build on prior knowledge within and across sessions.



Short-Term Memory

Conversation history within a single task. Auto-managed in the agent's context window.



Long-Term Memory

Persisted across sessions using vector stores (Chroma, Pinecone). Agents recall previous run insights.



Entity Memory

Tracks specific entities — companies, people, products — encountered during task execution.



Contextual Passing

Task output automatically feeds into the next task as context. The backbone of sequential workflows.

Guardrails & Best Practices



Limit Delegation

Only manager agents get `allow_delegation=True`. Unlimited delegation → infinite loops.

⚠ Critical



Focus Your Prompts

Specific role + goal + backstory = dramatically better outputs. Vague = garbage in.

✅ Key Tip



Add a Reviewer Agent

Always include a final validation step. Never ship raw first-agent output to production.

✅ Best



Control API Costs

Set `max_iter` (default=15) and `max_rpm` to prevent runaway calls and budget explosions.

💰 Cost



Secure Your Data

Never embed API keys or PII in backstories or task descriptions. Use env variables.

🔒 Security

Taking CrewAI to the Next Level



Custom Tools

Decorate any Python function with `@tool` to create agent capabilities — APIs, databases, anything.

```
@tool
def my_search(q: str) -> str:
    return search_api(q)
```



Multiple LLMs

Each agent can use a different model. Mix GPT-4o for research, Claude for writing, Llama for tasks.

```
agent = Agent(llm='claude-opus-4-5')
writer = Agent(llm='gpt-4o')
```



RAG Integration

Connect Chroma, Pinecone, or Weaviate for knowledge-grounded, hallucination-resistant agents.

```
from crewai_tools import RagTool
tool = RagTool(config=rag_config)
```



Human-in-the-Loop

Pause crew execution for human review, approval, or input before proceeding to critical next steps.

```
task = Task(
    human_input=True, ...)
```

CrewAI vs The Landscape

Feature	CrewAI	LangGraph	AutoGen	LlamaIndex
Role-based agents	✓ Native	⚠ Manual	⚠ Manual	✗ No
Task orchestration	✓ Powerful	✓ Powerful	⚠ Basic	⚠ Basic
Learning curve	● Low	● High	● Medium	● Medium
RAG integration	✓ Yes	✓ Yes	⚠ Partial	✓ Native
Human-in-the-loop	✓ Yes	✓ Yes	✓ Yes	⚠ Limited
Production ready (2025)	✓ Yes	✓ Yes	⚠ Beta	✓ Yes
Best for beginners	✓ Yes	✗ No	⚠ Partial	✗ No

CrewAI in Production Today



AWS Cloud Assessment

- Cloud Scanner
- Security Analyzer
- Report Writer

✓ Multi-account AWS audit report generated in minutes vs. weeks



Incident Management

- Monitor Agent
- Diagnosis Agent
- Remediation Agent

✓ P1 incidents detected, diagnosed & resolved autonomously 24/7



Proposal Generation

- Req. Analyst
- Solution Architect
- Proposal Writer

✓ RFP responses in hours — competing for contracts 10× faster



Automated Consulting

- Data Analyst
- Domain Expert
- Consultant Writer

✓ End-to-end consulting deliverables from raw data inputs

What You've Learned Today

01

CrewAI is Python's premier framework for orchestrating teams of autonomous AI agents

**02**

Agents = AI specialists with role, goal, backstory, tools, and memory

**03**

Tasks = Work briefs with expected outputs, assigned agents, and context sharing

**04**

Processes control HOW agents work — Sequential, Hierarchical, or Consensual

**05**

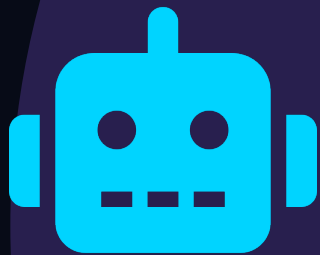
Crew = The AI company — orchestrating everything toward a unified mission

**06**

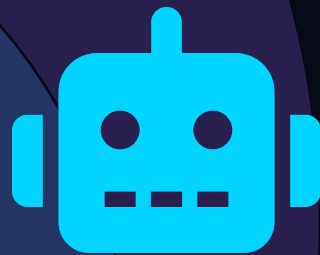
Best practices: focused backstories, validator agents, delegation limits, and cost controls



"One AI is powerful."



**"A well-coordinated team
of AI specialists
is transformational."**



Start today: github.com/crewAIInc/crewAI | docs.crewai.com | `pip install crewai`



Questions?

Ready to Build Your Own AI Dream Team?

Docs

`docs.crewai.com`

GitHub

`crewAIInc/crewAI`

Discord

`crewai.io/community`

Install

`pip install crewai`

Thank you! 🙏 Your AI Dream Team awaits.